

## DEVOPS MASTER COURSE

# VOLUME 2

## Bash Shell Scripting and DevOps Automation

Absolute Beginner to Production-Ready Engineer

### Learning Promise

This volume teaches why each Bash feature exists, how it behaves, how companies use it, how it fails, and how to practise it safely. Completion requires hands-on evidence, not only reading.

Edition 1.0 | 30 Units | Labs | Projects | Interview Preparation

# How to Use This Book

Study one unit at a time. Read the explanation, type every example yourself, complete the lab, answer the knowledge check without notes, and then mark the unit completed. Do not copy and paste commands you do not understand. Use a Linux virtual machine or a safe local laboratory, never a production server.

- ☐ Create a dedicated Linux laboratory.
- ☐ Use a normal user account and sudo only when required.
- ☐ Keep every exercise in a Git repository.
- ☐ Record errors and their fixes in your work notes.
- ☐ Complete each capstone and retain the code as portfolio evidence.

## Recommended Laboratory

| Component        | Recommendation                                                         |
|------------------|------------------------------------------------------------------------|
| Operating system | Ubuntu Server LTS in VirtualBox, VMware, WSL2, or a cloud free-tier VM |
| Account          | Non-root user with sudo                                                |
| Core tools       | bash, git, curl, jq, shellcheck, shfmt, bats, rsync                    |
| Workspace        | ~/devops-labs/bash-volume-2                                            |
| Safety           | No real production credentials or systems                              |

## Master Completion Criteria

- ☐ All 30 units marked completed.
- ☐ Every hands-on lab reproduced without copying the solution.
- ☐ Both capstone projects stored in Git with README and tests.
- ☐ ShellCheck and shfmt pass on final projects.
- ☐ At least 80% score in the final theory interview.
- ☐ At least 80% score in the practical examination.

# Course Map

1. Unit 1 - Shell, Bash and the Automation Mindset
2. Unit 2 - Script Structure, Comments and Execution Modes
3. Unit 3 - Variables, Quoting and Environment
4. Unit 4 - User Input, Arguments and Options
5. Unit 5 - Arithmetic and Numeric Validation
6. Unit 6 - Conditions, Tests and Decision Making
7. Unit 7 - case Statements and State Handling
8. Unit 8 - Loops and Repetition
9. Unit 9 - Functions, Scope and Reusable Design
10. Unit 10 - Arrays and Associative Arrays
11. Unit 11 - Redirection, Pipes and File Descriptors
12. Unit 12 - Text Processing with grep, sed, awk, cut, sort and uniq
13. Unit 13 - Regular Expressions and Pattern Matching
14. Unit 14 - Exit Codes, Error Handling and Strict Mode
15. Unit 15 - Traps, Signals and Cleanup
16. Unit 16 - Logging, Timestamps and Observability
17. Unit 17 - Files, Directories and Safe Configuration Changes
18. Unit 18 - Network Automation with curl, DNS and Timeouts
19. Unit 19 - JSON, YAML and APIs with jq and yq
20. Unit 20 - Scheduling with cron and systemd Timers
21. Unit 21 - Remote Execution, SSH and Fleet Safety
22. Unit 22 - Packaging, Portability and Dependencies
23. Unit 23 - Security for Shell Scripts
24. Unit 24 - Testing Bash with ShellCheck, shfmt and Bats
25. Unit 25 - Debugging and Performance Analysis
26. Unit 26 - Backup, Retention and Recovery Automation
27. Unit 27 - Deployment Automation Patterns
28. Unit 28 - CI/CD Integration for Shell Scripts
29. Unit 29 - Capstone Project: Production Health and Readiness Toolkit
30. Unit 30 - Capstone Project: Backup and Restore Orchestrator

# Unit 1 - Shell, Bash and the Automation Mindset

## Why This Matters

Before writing scripts, you must understand what the shell is, why Bash exists, and how commands become repeatable automation. This prevents the common beginner mistake of memorising syntax without understanding execution.

## Core Concepts

| Concept            | Explanation                                                                                                               |
|--------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Shell</b>       | A command interpreter that accepts input, expands it, invokes programs, and returns status information.                   |
| <b>Bash</b>        | The Bourne Again Shell. It is widely available on Linux servers and is commonly used for operational automation.          |
| <b>Script</b>      | A text file containing commands and shell language constructs that are executed in sequence.                              |
| <b>Interpreter</b> | The program that reads and executes a script. The shebang selects it.                                                     |
| <b>Exit status</b> | Every command returns an integer status. Zero usually means success; non-zero indicates a failure or special condition.   |
| <b>Idempotence</b> | Running an automation repeatedly should produce the intended state without causing duplicate or destructive side effects. |

## Worked Examples

### Your first script

```
#!/usr/bin/env bash
echo "Hello from Bash"
```

### Run it safely

```
chmod +x hello.sh
./hello.sh
echo "$?"
```

### Inspect Bash

```
echo "$BASH_VERSION"
command -v bash
getent passwd "$USER"
```

## How Companies Use It

Companies use Bash for bootstrap scripts, deployment helpers, health checks, backup jobs, CI/CD glue code, operational runbooks, and emergency remediation. Bash is excellent for orchestrating command-line tools. Large application logic is normally better implemented in Python, Go, Java, or another general-purpose language.

**Hands-on Lab: Build and inspect your first script**

Objective: Create, execute and validate a script with a predictable exit code.

Steps:

31. Create ~/devops-labs/bash and enter it.
32. Create hello.sh with a portable Bash shebang.
33. Print your username, hostname and current directory.
34. Make the script executable and run it.
35. Print the script exit status using echo "\$?".

Verification: The script prints correct values and returns exit status 0.

**Common Mistakes and Risks**

- Running scripts copied from the internet without reading them.
- Using sudo for every command.
- Assuming a script succeeded because it printed output.
- Writing complex business logic in Bash when another language would be clearer.

**Knowledge Check****1. What does a shell do?**

Answer: \_\_\_\_\_

**2. Why is Bash called an interpreter?**

Answer: \_\_\_\_\_

**3. What is a shebang?**

Answer: \_\_\_\_\_

**4. What does exit status 0 usually mean?**

Answer: \_\_\_\_\_

**5. Why is idempotence important?**

Answer: \_\_\_\_\_

**Interview Questions**

36. What is the difference between a shell and a terminal?
37. Why might you use #!/usr/bin/env bash instead of #!/bin/bash?
38. When should Bash not be used?

**Unit Skills Checklist**

- ☐ Explain shell, terminal, Bash and script in your own words.
- ☐ Create and execute a script without help.
- ☐ Check and interpret an exit code.

**Completion Gate**

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 1 - Shell, Bash and the Automation Mindset completed.

# Unit 2 - Script Structure, Comments and Execution Modes

## Why This Matters

Professional scripts are readable, predictable and easy to troubleshoot. Structure and execution mode determine how Bash interprets your file.

## Core Concepts

| Concept                      | Explanation                                                                |
|------------------------------|----------------------------------------------------------------------------|
| <b>Shebang</b>               | The first line that identifies the interpreter.                            |
| <b>Comments</b>              | Lines beginning with # are ignored, except the shebang.                    |
| <b>Interactive shell</b>     | A shell attached to a user terminal.                                       |
| <b>Non-interactive shell</b> | A shell executing a script or pipeline without conversational input.       |
| <b>Source</b>                | Execute a file in the current shell using source file or . file.           |
| <b>Subshell</b>              | A child shell with its own environment; changes may not affect the parent. |

## Worked Examples

### Recommended skeleton

```
#!/usr/bin/env bash
set -Eeuo pipefail

main() {
    printf '%s\n' "Starting"
}

main "$@"
```

### Execute versus source

```
./variables.sh
source ./variables.sh
. ./variables.sh
```

### Syntax check

```
bash -n script.sh
bash -x script.sh
```

## How Companies Use It

In production repositories, scripts usually contain a header, strict-mode decision, functions, a main entry point, validation, logging and explicit exit codes. Teams use shellcheck and automated tests to enforce quality.

### Hands-on Lab: Compare execution and sourcing

Objective: Observe the difference between a child shell and the current shell.

Steps:

39. Create setenv.sh containing export DEMO\_ENV=production.
40. Run ./setenv.sh and then echo "\$DEMO\_ENV".
41. Run source ./setenv.sh and then echo "\$DEMO\_ENV".

42. Explain why the results differ.

Verification: The variable is visible only after sourcing the file into the current shell.

### Common Mistakes and Risks

- Sourcing untrusted scripts.
- Using set -e without understanding exceptions.
- Placing executable statements before function definitions without a clear structure.

### Knowledge Check

1. What is the role of comments?

Answer: \_\_\_\_\_

2. What is the difference between executing and sourcing?

Answer: \_\_\_\_\_

3. What does bash -n do?

Answer: \_\_\_\_\_

4. What does bash -x do?

Answer: \_\_\_\_\_

5. Why use a main function?

Answer: \_\_\_\_\_

### Interview Questions

43. What risks can set -e introduce?
44. What is a subshell?
45. How do environment changes behave across parent and child processes?

### Unit Skills Checklist

- ☐ Explain execute versus source.
- ☐ Use bash -n and bash -x.
- ☐ Create a structured script with main().

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 2 - Script Structure, Comments and Execution Modes completed.

# Unit 3 - Variables, Quoting and Environment

## Why This Matters

Most shell bugs come from incorrect quoting, word splitting or accidental glob expansion. Mastering variables and quoting is fundamental.

## Core Concepts

| Concept              | Explanation                                                                 |
|----------------------|-----------------------------------------------------------------------------|
| Variable assignment  | No spaces around =. Example: name="Sunny".                                  |
| Parameter expansion  | Use \$name or preferably \${name} when boundaries are ambiguous.            |
| Environment variable | A variable exported to child processes.                                     |
| Single quotes        | Preserve text literally; variables are not expanded.                        |
| Double quotes        | Allow parameter and command expansion while preventing most word splitting. |
| Command substitution | Capture command output with \$(command).                                    |
| Readonly             | Prevent accidental reassignment.                                            |

## Worked Examples

### Safe variables

```
name="Sunny"
city="Chirala"
printf 'Name: %s, City: %s\n' "$name" "$city"
```

### Quoting comparison

```
value="hello world"
printf '%s\n' "$value"
printf '%s\n' '$value'
```

### Environment

```
export APP_ENV="production"
env | grep '^APP_ENV='
```

### Defaults

```
region="${AWS_REGION:-ap-south-1}"
required="${API_TOKEN:?API_TOKEN must be set}"
```

## How Companies Use It

CI/CD platforms provide credentials, branch names, build IDs and environment names through environment variables. Professional scripts validate required variables and never print secrets.

### Hands-on Lab: Quote filenames safely

Objective: Create and process filenames containing spaces.

Steps:

46. Create a directory named demo files.
47. Store its name in a variable.
48. Create a file called deployment notes.txt inside it.

49. List and read the file using correctly quoted variables.
50. Repeat without quotes in a safe sandbox and observe the error.

Verification: The quoted version works reliably and the learner can explain the unquoted failure.

### Common Mistakes and Risks

- Unquoted variables.
- Using echo for complex or option-like data.
- Storing secrets directly inside source-controlled scripts.
- Confusing local shell variables with exported environment variables.

### Knowledge Check

#### 1. Why are spaces forbidden around variable assignment?

Answer: \_\_\_\_\_

#### 2. What is the difference between single and double quotes?

Answer: \_\_\_\_\_

#### 3. Why should "\$variable" usually be quoted?

Answer: \_\_\_\_\_

#### 4. What does export do?

Answer: \_\_\_\_\_

#### 5. How does \${VAR:-default} work?

Answer: \_\_\_\_\_

### Interview Questions

51. Explain word splitting and pathname expansion.
52. How do you require an environment variable?
53. How do you prevent secrets from appearing in logs?

### Unit Skills Checklist

- ☐ Use safe variable assignments.
- ☐ Explain all three quoting contexts.
- ☐ Validate required environment variables.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 3 - Variables, Quoting and Environment completed.

# Unit 4 - User Input, Arguments and Options

## Why This Matters

Scripts become reusable when they accept validated inputs rather than depending on hard-coded values.

## Core Concepts

| Concept                      | Explanation                                                      |
|------------------------------|------------------------------------------------------------------|
| <b>read</b>                  | Read input from standard input.                                  |
| <b>Positional parameters</b> | \$1, \$2 and so on contain arguments.                            |
| <b>Special parameters</b>    | #, \$@, \$*, \$\$ and \$? provide script context.                |
| <b>shift</b>                 | Remove the first positional argument and renumber the remainder. |
| <b>getopts</b>               | Parse short command-line options reliably.                       |
| <b>Usage function</b>        | Explain accepted syntax and return a meaningful error status.    |

## Worked Examples

### Arguments

```
#!/usr/bin/env bash
name="${1:-Guest}"
printf 'Hello %s\n' "$name"
```

### All arguments

```
for arg in "$@"; do
    printf 'Argument: %s\n' "$arg"
done
```

### Input prompt

```
read -r -p "Environment: " environment
printf 'Selected: %s\n' "$environment"
```

### getopts

```
while getopts ":e:v" opt; do
    case "$opt" in
        e) environment="$OPTARG" ;;
        v) verbose=true ;;
        :) printf 'Option -%s needs a value\n' "$OPTARG" >&2; exit 2 ;;
        \?) printf 'Unknown option: -%s\n' "$OPTARG" >&2; exit 2 ;;
    esac
done
```

## How Companies Use It

Deployment scripts accept environment, release version and dry-run options. Mature tools have stable interfaces, clear help text and backwards-compatible argument handling.

### Hands-on Lab: Create a deployment argument parser

Objective: Build a script that accepts environment and version.

Steps:

54. Accept -e for environment and -v for version.
55. Reject missing values and unknown options.
56. Allow only dev, test and prod environments.
57. Print a clear usage message.
58. Return status 2 for usage errors.

Verification: Valid input prints the deployment plan; invalid input is rejected with a useful message and status 2.

### Common Mistakes and Risks

- Using read in unattended CI jobs.
- Ignoring missing arguments.
- Treating all invalid inputs as exit status 1 without documenting meanings.

### Knowledge Check

#### 1. What does "\$@" represent?

Answer: \_\_\_\_\_

#### 2. Why use read -r?

Answer: \_\_\_\_\_

#### 3. What does shift do?

Answer: \_\_\_\_\_

#### 4. Why should usage errors return a non-zero status?

Answer: \_\_\_\_\_

#### 5. What is getopts used for?

Answer: \_\_\_\_\_

### Interview Questions

59. Difference between "\$@" and "\$\*"?
60. How would you support --long-options?
61. How do you design a backwards-compatible CLI?

### Unit Skills Checklist

- ☐ Parse positional arguments.
- ☐ Use getopts.
- ☐ Validate inputs and show usage.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 4 - User Input, Arguments and Options completed.

# Unit 5 - Arithmetic and Numeric Validation

## Why This Matters

DevOps automation frequently calculates thresholds, retention windows, retry counts and resource utilisation.

## Core Concepts

| Concept                     | Explanation                                                                                                                    |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Arithmetic expansion</b> | <code>\$(( expression ))</code> performs integer arithmetic.                                                                   |
| <b>Arithmetic command</b>   | <code>(( expression ))</code> evaluates arithmetic and returns a status.                                                       |
| <b>Operators</b>            | <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code> , <code>**</code> and comparison operators. |
| <b>Increment</b>            | <code>((count++))</code> or <code>((++count))</code> ; behaviour matters under <code>set -e</code> .                           |
| <b>Integer limitation</b>   | Bash arithmetic is integer-based; use <code>awk</code> or <code>bc</code> for decimals.                                        |
| <b>Validation</b>           | Check numeric input before calculating.                                                                                        |

## Worked Examples

### Calculation

```
used=75
total=100
percentage=$(( used * 100 / total ))
printf 'Used: %d%%\n' "$percentage"
```

### Numeric validation

```
value="${1:-}"
if [[ ! "$value" =~ ^[0-9]+$ ]]; then
    printf 'Expected a non-negative integer\n' >&2
    exit 2
fi
```

### Decimal with awk

```
awk -v used=75 -v total=128 'BEGIN { printf "%.2f\n", used*100/total }'
```

## How Companies Use It

Monitoring scripts calculate disk thresholds and retry budgets. Production scripts protect against division by zero, invalid metrics and overflow assumptions.

### Hands-on Lab: Disk threshold calculator

Objective: Calculate an alert state from numeric input.

Steps:

62. Accept `used` and `total` values as arguments.
63. Validate both as positive integers.
64. Reject `total=0`.
65. Calculate integer percentage.
66. Print OK below 80, WARNING from 80 to 89, and CRITICAL from 90.

Verification: Tests for 79, 80, 89 and 90 produce the expected states.

## Common Mistakes and Risks

- Division by zero.
- Assuming command output is always numeric.
- Missing boundary tests.
- Using `expr` unnecessarily in modern Bash.

## Knowledge Check

### 1. How is arithmetic expansion written?

Answer: \_\_\_\_\_

### 2. What happens with integer division?

Answer: \_\_\_\_\_

### 3. How can decimals be handled?

Answer: \_\_\_\_\_

### 4. Why validate before calculating?

Answer: \_\_\_\_\_

### 5. What does modulo return?

Answer: \_\_\_\_\_

## Interview Questions

67. Why can `((count++))` interact badly with `set -e`?
68. How would you handle floating-point thresholds?
69. What boundary tests are important?

## Unit Skills Checklist

- ☐ Perform integer arithmetic.
- ☐ Validate numeric input.
- ☐ Test threshold boundaries.

## Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 5 - Arithmetic and Numeric Validation completed.

# Unit 6 - Conditions, Tests and Decision Making

## Why This Matters

Conditions allow scripts to make safe decisions based on files, strings, numbers and command outcomes.

## Core Concepts

| Concept                     | Explanation                                                                  |
|-----------------------------|------------------------------------------------------------------------------|
| <b>if/elif/else</b>         | Choose among branches.                                                       |
| <b>test and [ ]</b>         | Portable test syntax; spacing is mandatory.                                  |
| <b>[[ ]]</b>                | Bash conditional expression with safer string behaviour and pattern support. |
| <b>File tests</b>           | -e, -f, -d, -r, -w, -x, -s and newer-than tests.                             |
| <b>String tests</b>         | -z, -n, ==, != and pattern matching.                                         |
| <b>Numeric tests</b>        | -eq, -ne, -lt, -le, -gt, -ge.                                                |
| <b>Command-as-condition</b> | if command; then uses the command exit status directly.                      |

## Worked Examples

### File decision

```
config="/etc/myapp/config.yml"
if [[ -r "$config" ]]; then
    printf 'Config is readable\n'
else
    printf 'Config missing or unreadable\n' >&2
    exit 1
fi
```

### Command decision

```
if systemctl is-active --quiet nginx; then
    printf 'nginx is active\n'
else
    printf 'nginx is not active\n'
fi
```

### Pattern

```
branch="feature/login"
if [[ "$branch" == feature/* ]]; then
    printf 'Feature branch\n'
fi
```

## How Companies Use It

Health checks and deployment gates use command exit statuses rather than parsing human-readable output. This is more stable and language-independent.

### Hands-on Lab: Build a pre-deployment gate

Objective: Validate conditions before a pretend deployment.

Steps:

70. Require a readable configuration file.
71. Require at least one argument containing a release version.

- 72. Allow only branches main or release/\*.
- 73. Check that curl exists using command -v.
- 74. Print all failed checks and return non-zero.

Verification: All invalid cases are blocked; valid input reaches the deployment-ready message.

### Common Mistakes and Risks

- Using = for numeric comparison in [ ].
- Unquoted variables inside [ ].
- Parsing formatted output when an exit code exists.
- Failing at the first validation when showing all errors would help the user.

### Knowledge Check

#### 1. Why is spacing important in [ ]?

Answer: \_\_\_\_\_

#### 2. When is [[ ]] preferable?

Answer: \_\_\_\_\_

#### 3. How do you test whether a file is executable?

Answer: \_\_\_\_\_

#### 4. Why use a command directly as an if condition?

Answer: \_\_\_\_\_

#### 5. What is the difference between string and numeric comparison?

Answer: \_\_\_\_\_

### Interview Questions

- 75. What does [[ "\$x" == pattern\* ]] do?
- 76. Why is parsing systemctl status output fragile?
- 77. How would you aggregate multiple validation failures?

### Unit Skills Checklist

- ☐ Write file, string and numeric conditions.
- ☐ Use command exit status as a condition.
- ☐ Implement a deployment gate.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 6 - Conditions, Tests and Decision Making completed.

# Unit 7 - case Statements and State Handling

## Why This Matters

case is clearer than long if chains when matching commands, environments, operating systems or states.

## Core Concepts

| Concept        | Explanation                                                    |
|----------------|----------------------------------------------------------------|
| case syntax    | Match one value against ordered patterns.                      |
| Patterns       | Use   for alternatives and * as a wildcard.                    |
| Default branch | The * pattern handles unknown values.                          |
| Fall-through   | ;& and ;;& exist in Bash but should be used deliberately.      |
| State machine  | Represent operational states and valid transitions explicitly. |

## Worked Examples

### Environment selection

```
case "${1:-}" in
  dev|test) replicas=1 ;;
  staging) replicas=2 ;;
  prod) replicas=4 ;;
  *) printf 'Usage: %s {dev|test|staging|prod}\n' "$0" >&2; exit 2 ;;
esac
printf 'Replicas: %d\n' "$replicas"
```

### OS detection

```
source /etc/os-release
case "$ID" in
  ubuntu|debian) package_manager=apt ;;
  rhel|rocky|almalinux|fedora) package_manager=dnf ;;
  *) package_manager=unknown ;;
esac
```

## How Companies Use It

Cross-distribution bootstrap scripts detect operating systems and select package commands. Release tools model statuses such as pending, deploying, verified and rolled\_back.

### Hands-on Lab: Create a service action tool

Objective: Use case to support start, stop, restart and status.

Steps:

78. Accept service name and action.
79. Validate that both arguments are present.
80. Use case for the action.
81. Run systemctl only after printing the planned command.
82. Add a --dry-run mode that never changes the service.

Verification: All actions map correctly and unknown actions show usage.

### Common Mistakes and Risks

- Missing default patterns.

- Combining unrelated responsibilities into one enormous case block.
- Executing privileged actions before validation.

### Knowledge Check

**1. When is case clearer than if?**

Answer: \_\_\_\_\_

**2. How do you match alternatives?**

Answer: \_\_\_\_\_

**3. Why include a default branch?**

Answer: \_\_\_\_\_

**4. What is a state transition?**

Answer: \_\_\_\_\_

**5. Why is dry-run valuable?**

Answer: \_\_\_\_\_

### Interview Questions

83. How can case be used for OS portability?
84. What are ;& and ;;&?
85. How would you validate allowed state transitions?

### Unit Skills Checklist

- ☐ Use case for command dispatch.
- ☐ Implement a default error branch.
- ☐ Explain dry-run behaviour.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 7 - case Statements and State Handling completed.

# Unit 8 - Loops and Repetition

## Why This Matters

Loops automate repeated work across files, hosts, services and API results. Safe loops must handle spaces, failures and empty inputs.

## Core Concepts

| Concept                   | Explanation                                                             |
|---------------------------|-------------------------------------------------------------------------|
| <b>for loop</b>           | Iterate over a known list or generated sequence.                        |
| <b>while loop</b>         | Repeat while a condition or command succeeds.                           |
| <b>until loop</b>         | Repeat until a condition becomes true.                                  |
| <b>break and continue</b> | Control loop flow.                                                      |
| <b>C-style loop</b>       | Useful for counters: for ((i=0; i<3; i++)).                             |
| <b>Read loop</b>          | while IFS= read -r line; do ...; done is the safe text-reading pattern. |

## Worked Examples

### Arguments loop

```
for host in "$@"; do
    printf 'Checking %s\n' "$host"
done
```

### Read file safely

```
while IFS= read -r host || [[ -n "$host" ]]; do
    [[ -z "$host" || "$host" == \#* ]] && continue
    printf 'Host: %s\n' "$host"
done < hosts.txt
```

### Retry loop

```
attempt=1
max=5
until curl -fsS http://localhost:8080/health >/dev/null; do
    (( attempt >= max )) && exit 1
    sleep "$attempt"
    ((++attempt))
done
```

## How Companies Use It

Operations teams loop through inventories, but large fleets are normally handled with Ansible, Kubernetes, cloud APIs or orchestration platforms rather than ad-hoc SSH loops.

### Hands-on Lab: Create a resilient health checker

Objective: Check multiple URLs from a file.

Steps:

86. Create urls.txt with comments and blank lines.
87. Read it safely line by line.
88. Use curl with connection and total timeouts.
89. Count successes and failures.

90. Return non-zero if any URL fails.

Verification: The summary counts are correct and the script exit status reflects overall health.

### Common Mistakes and Risks

- for item in \$(cat file).
- No timeout on network commands.
- Ignoring partial failures.
- Infinite retries without a maximum.

### Knowledge Check

1. When should you use while instead of for?

Answer: \_\_\_\_\_

2. Why use IFS= read -r?

Answer: \_\_\_\_\_

3. What do break and continue do?

Answer: \_\_\_\_\_

4. How do you avoid an infinite loop?

Answer: \_\_\_\_\_

5. How should a batch script report partial failures?

Answer: \_\_\_\_\_

### Interview Questions

91. What is the pipeline subshell problem with while read?
92. How would you limit concurrency?
93. Why are shell loops poor for very large fleets?

### Unit Skills Checklist

- ☐ Write for, while and until loops.
- ☐ Read files safely.
- ☐ Implement bounded retries.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 8 - Loops and Repetition completed.

# Unit 9 - Functions, Scope and Reusable Design

## Why This Matters

Functions make scripts modular, testable and understandable. Good functions have clear inputs, outputs and side effects.

## Core Concepts

| Concept                      | Explanation                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Function definition</b>   | <code>name() { commands; }</code>                                                                  |
| <b>Arguments</b>             | Functions receive their own positional parameters.                                                 |
| <b>Return status</b>         | <code>return</code> sets a numeric status; it does not return arbitrary text.                      |
| <b>Output as data</b>        | A function can print a value for command substitution, but logs should go to <code>stderr</code> . |
| <b>local variables</b>       | Use <code>local</code> to avoid accidental global changes.                                         |
| <b>Single responsibility</b> | Each function should do one coherent job.                                                          |

## Worked Examples

### Validation function

```
require_command() {
  local command_name="$1"
  command -v "$command_name" >/dev/null 2>&1 || {
    printf 'Missing command: %s\n' "$command_name" >&2
    return 1
  }
}
```

### Value function

```
get_timestamp() { date -u +%Y-%m-%dT%H:%M:%SZ; }
timestamp="$(get_timestamp)"
```

### Main design

```
main() {
  require_command curl
  check_health "${1:-http://localhost:8080/health}"
}
main "$@"
```

## How Companies Use It

Company script libraries centralise logging, validation and cloud authentication helpers. However, sourcing large mutable libraries can create hidden coupling, so teams keep interfaces small and versioned.

### Hands-on Lab: Refactor a monolithic script

Objective: Split validation, logging and work into functions.

Steps:

94. Take an earlier health-check script.
95. Create `log_info`, `log_error`, `require_command` and `check_url` functions.
96. Use `local` variables in every function.

97. Create main and pass "\$@".

98. Ensure failures propagate to the final exit status.

Verification: The script has no accidental global variables and functions can be tested independently.

### Common Mistakes and Risks

- Using global variables everywhere.
- Mixing logs and machine-readable output on stdout.
- Functions that silently exit the whole script.
- Deeply nested functions with unclear contracts.

### Knowledge Check

#### 1. What does return do?

Answer: \_\_\_\_\_

#### 2. Why use local?

Answer: \_\_\_\_\_

#### 3. How do functions receive arguments?

Answer: \_\_\_\_\_

#### 4. Why should logs go to stderr when stdout is data?

Answer: \_\_\_\_\_

#### 5. What is single responsibility?

Answer: \_\_\_\_\_

### Interview Questions

99. How do function side effects affect testing?

100. How can you mock external commands?

101. When should a helper become a separate executable?

### Unit Skills Checklist

- ☐ Create reusable functions.
- ☐ Use local scope.
- ☐ Separate stdout data from stderr logs.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 9 - Functions, Scope and Reusable Design completed.

# Unit 10 - Arrays and Associative Arrays

## Why This Matters

Arrays help manage lists of services, hosts, files and configuration mappings without unsafe string parsing.

## Core Concepts

| Concept           | Explanation                                                                                                      |
|-------------------|------------------------------------------------------------------------------------------------------------------|
| Indexed array     | Ordered values referenced by numeric index.                                                                      |
| Associative array | Key-value mapping available in Bash 4+.                                                                          |
| Expansion         | "\${array[@]}" preserves elements safely.                                                                        |
| Length            | \${#array[@]} returns element count.                                                                             |
| Append            | array+=("value").                                                                                                |
| Compatibility     | Associative arrays may not be available in older Bash versions, notably the default Bash on older macOS systems. |

## Worked Examples

### Indexed array

```
services=(nginx ssh cron)
for service in "${services[@]}; do
    printf '%s\n' "$service"
done
```

### Associative array

```
declare -A ports=( [http]=80 [https]=443 [ssh]=22 )
for name in "${!ports[@]}; do
    printf '%s=%s\n' "$name" "${ports[$name]}"
done
```

### Append

```
failed=()
failed+=("api-service")
printf 'Failures: %d\n' "${#failed[@]}"
```

## How Companies Use It

Deployment scripts use arrays for components and associative arrays for metadata. For complex nested data, JSON with jq or a higher-level language is usually better.

### Hands-on Lab: Service status dashboard

Objective: Collect service results into arrays.

Steps:

102. Define an array of three services available on your system.
103. Check each service with `systemctl is-active --quiet`.
104. Store healthy and unhealthy names in separate arrays.
105. Print counts and names safely.
106. Return non-zero when unhealthy services exist.

Verification: The script handles zero, one or many unhealthy services without malformed output.

## Common Mistakes and Risks

- Storing lists as space-separated strings.
- Unquoted array expansion.
- Using Bash arrays for deeply nested configuration.

## Knowledge Check

### 1. How do you expand all array elements safely?

Answer: \_\_\_\_\_

### 2. How do you get array length?

Answer: \_\_\_\_\_

### 3. What is an associative array?

Answer: \_\_\_\_\_

### 4. How do you append an element?

Answer: \_\_\_\_\_

### 5. When should JSON replace arrays?

Answer: \_\_\_\_\_

## Interview Questions

107. Difference between `${array[*]}` and `${array[@]}` when quoted?

108. How do you iterate over associative keys?

109. What portability concerns exist?

## Unit Skills Checklist

- ☐ Create indexed and associative arrays.
- ☐ Iterate safely.
- ☐ Choose an appropriate data representation.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 10 - Arrays and Associative Arrays completed.

# Unit 11 - Redirection, Pipes and File Descriptors

## Why This Matters

Automation depends on correctly controlling standard input, standard output and standard error.

## Core Concepts

| Concept              | Explanation                                        |
|----------------------|----------------------------------------------------|
| <b>stdin</b>         | File descriptor 0; input stream.                   |
| <b>stdout</b>        | File descriptor 1; normal output.                  |
| <b>stderr</b>        | File descriptor 2; diagnostics and errors.         |
| <b>Redirection</b>   | >, >>, <, 2>, 2>&1 and &>.                         |
| <b>Pipeline</b>      | Connect stdout of one process to stdin of another. |
| <b>tee</b>           | Write output to a file and continue the pipeline.  |
| <b>Here-document</b> | Provide multi-line input using <<EOF.              |
| <b>Here-string</b>   | Provide a single string using <<<.                 |

## Worked Examples

### Separate logs

```
command >output.log 2>error.log
```

### Combined log

```
command >combined.log 2>&1
```

### Pipeline status

```
set -o pipefail
producer | transformer | consumer
```

### Tee

```
journalctl -u nginx --since today | tee nginx-today.log | grep -i error
```

### Here-document

```
cat > app.conf <<'EOF'
APP_ENV=production
LOG_LEVEL=info
EOF
```

## How Companies Use It

CI systems capture stdout and stderr differently. Machine-readable output belongs on stdout, while diagnostics belong on stderr. This allows tools to compose reliably.

### Hands-on Lab: Build a log collector

Objective: Capture command output predictably.

Steps:

- 110.Run a successful command and a failing command.
- 111.Send normal output to results.log.
- 112.Send errors to errors.log.

113. Create a combined timestamped transcript with tee.

114. Verify the final script status is non-zero if collection fails.

Verification: Each stream contains the expected data and failure status is preserved.

### Common Mistakes and Risks

- Redirecting stderr before stdout without understanding order.
- Pipelines hiding failures.
- Writing secrets to CI logs.
- Using temporary files when a pipeline is clearer.

### Knowledge Check

**1. What are file descriptors 0, 1 and 2?**

Answer: \_\_\_\_\_

**2. What is the difference between > and >>?**

Answer: \_\_\_\_\_

**3. What does 2>&1 mean?**

Answer: \_\_\_\_\_

**4. Why use pipefail?**

Answer: \_\_\_\_\_

**5. What does tee do?**

Answer: \_\_\_\_\_

### Interview Questions

115. Why does redirection order matter?

116. How do you create an additional file descriptor?

117. What is SIGPIPE?

### Unit Skills Checklist

- ☐ Control stdout and stderr separately.
- ☐ Use pipefail.
- ☐ Create a here-document safely.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 11 - Redirection, Pipes and File Descriptors completed.

# Unit 12 - Text Processing with grep, sed, awk, cut, sort and uniq

## Why This Matters

Text processing is a core DevOps skill because Linux commands, logs and configuration files often produce line-oriented data.

## Core Concepts

| Concept              | Explanation                                                            |
|----------------------|------------------------------------------------------------------------|
| <b>grep</b>          | Filter lines by text or regular expression.                            |
| <b>sed</b>           | Stream editor for substitutions and line transformations.              |
| <b>awk</b>           | Field-oriented processing and reporting language.                      |
| <b>cut</b>           | Extract delimited fields or character ranges.                          |
| <b>sort and uniq</b> | Order data and count duplicates.                                       |
| <b>tr</b>            | Translate or delete characters.                                        |
| <b>xargs</b>         | Build command arguments from input; use null delimiters for filenames. |

## Worked Examples

### grep

```
grep -Rni --exclude-dir=.git 'timeout' .
```

### sed

```
sed -E 's/(password=).*\/\1REDACTED/' app.conf
```

### awk

```
awk '{count[$9]++} END {for (code in count) print code, count[code]}' access.log | sort -n
```

### cut/sort/uniq

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -nr
```

### Safe xargs

```
find . -type f -name '*.log' -print0 | xargs -0 grep -l 'ERROR'
```

## How Companies Use It

Incident responders use these tools for rapid analysis. Repeatable production analytics should often move to structured logs, SQL, Python or observability platforms.

### Hands-on Lab: Analyse an access log

Objective: Produce a small operational report.

Steps:

118. Create a sample access log containing IP, path and status code fields.
119. Count requests by status code with awk.
120. Show the top three client IPs.

121. Extract only 5xx lines with `grep`.

122. Write a redacted copy replacing IP addresses with REDACTED.

Verification: Counts match the sample data and the original file remains unchanged.

### Common Mistakes and Risks

- Parsing JSON with `grep`.
- Editing production files in-place without backups.
- Unsafe xargs with whitespace-containing filenames.
- Overly complex one-liners that nobody can maintain.

### Knowledge Check

#### 1. What is `grep` used for?

Answer: \_\_\_\_\_

#### 2. When is `awk` better than `cut`?

Answer: \_\_\_\_\_

#### 3. What does `sed -E` enable?

Answer: \_\_\_\_\_

#### 4. Why use `sort` before `uniq`?

Answer: \_\_\_\_\_

#### 5. Why are null delimiters important for filenames?

Answer: \_\_\_\_\_

### Interview Questions

123. How would you parse CSV safely?

124. What are the limitations of regex for structured formats?

125. Why should JSON be processed with `jq`?

### Unit Skills Checklist

- ☐ Use `grep`, `sed` and `awk` for appropriate tasks.
- ☐ Build a log summary.
- ☐ Recognise when structured tools are required.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 12 - Text Processing with `grep`, `sed`, `awk`, `cut`, `sort` and `uniq` completed.

# Unit 13 - Regular Expressions and Pattern Matching

## Why This Matters

Regular expressions support validation, filtering and extraction, but careless patterns can reject valid data or accept unsafe data.

## Core Concepts

| Concept                          | Explanation                                                      |
|----------------------------------|------------------------------------------------------------------|
| <b>Literal and metacharacter</b> | Some characters match themselves; others have special meaning.   |
| <b>Anchors</b>                   | ^ matches beginning and \$ matches end.                          |
| <b>Character class</b>           | [0-9], [[:space:]] and negated classes.                          |
| <b>Quantifiers</b>               | *, +, ? and {m,n} in extended regular expressions.               |
| <b>Grouping and alternation</b>  | (group) and a b.                                                 |
| <b>Bash regex</b>                | [[ string =~ regex ]] and BASH_REMATCH.                          |
| <b>Globs versus regex</b>        | Shell pathname patterns are not the same as regular expressions. |

## Worked Examples

### Validate environment

```
env_name="${1:-}"
if [[ "$env_name" =~ ^(dev|test|staging|prod)$ ]]; then
    printf 'Valid\n'
else
    printf 'Invalid\n' >&2
    exit 2
fi
```

### Capture version

```
version="v2.14.7"
if [[ "$version" =~ ^v([0-9]+)\.([0-9]+)\.([0-9]+)$ ]]; then
    major="${BASH_REMATCH[1]}"
fi
```

## How Companies Use It

Release pipelines validate semantic-looking version strings and branch naming rules. Security-sensitive validation should prefer allowlists and specialised parsers.

### Hands-on Lab: Validate release identifiers

Objective: Accept only controlled release formats.

Steps:

126. Accept an argument such as v1.2.3.
127. Use Bash regex to capture major, minor and patch.
128. Reject leading or trailing text.
129. Print captured components.
130. Add tests for valid and invalid examples.

Verification: Only the intended format is accepted and all three components are captured.

## Common Mistakes and Risks

- Using `*` when a precise pattern is possible.
- Confusing shell globs with regex.
- Using regex to parse complex JSON, YAML or HTML.

## Knowledge Check

### 1. What do `^` and `$` mean?

Answer: \_\_\_\_\_

### 2. What is a character class?

Answer: \_\_\_\_\_

### 3. How is a glob different from regex?

Answer: \_\_\_\_\_

### 4. What is `BASH_REMATCH`?

Answer: \_\_\_\_\_

### 5. Why prefer allowlists?

Answer: \_\_\_\_\_

## Interview Questions

131. What regex dialect does `[[ =~ ]]` use?
132. Why should the regex usually be unquoted on the right side?
133. When is a parser safer than regex?

## Unit Skills Checklist

- ☐ Write anchored validation patterns.
- ☐ Capture groups with `BASH_REMATCH`.
- ☐ Explain regex limits.

## Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 13 - Regular Expressions and Pattern Matching completed.

# Unit 14 - Exit Codes, Error Handling and Strict Mode

## Why This Matters

Production automation must fail clearly, preserve context and avoid silently continuing after critical errors.

## Core Concepts

| Concept            | Explanation                                                                        |
|--------------------|------------------------------------------------------------------------------------|
| Exit code contract | Document what success and failure statuses mean.                                   |
| set -e             | Exit on some unhandled failures; it has important exceptions.                      |
| set -u             | Treat unset variables as errors.                                                   |
| pipefail           | Make a pipeline fail when any component fails.                                     |
| ERR trap           | Run diagnostic logic when a command failure propagates.                            |
| Explicit checks    | Often clearer than relying only on set -e.                                         |
| Error context      | Include command, line, function and relevant identifiers without exposing secrets. |

## Worked Examples

### Strict baseline

```
set -Eeuo pipefail
```

### Error trap

```
trap 'rc=$?; printf "ERROR line=%s command=%q status=%s\n" "$LINENO" "$BASH_COMMAND" "$rc" >&2; exit "$rc"' ERR
```

### Explicit handling

```
if ! curl -fsS "$url" -o response.json; then
    printf 'Request failed: %s\n' "$url" >&2
    exit 1
fi
```

### Optional variable

```
debug="${DEBUG:-false}"
```

## How Companies Use It

Teams standardise error messages so CI logs and incident timelines are easy to understand. Strict mode is a tool, not a substitute for deliberate error handling and testing.

### Hands-on Lab: Harden a fragile script

Objective: Add predictable failure behaviour.

Steps:

134. Create a script with a failing pipeline and an unset variable.
135. Observe behaviour without strict options.
136. Enable -u and pipefail separately and record differences.
137. Add an ERR trap.

138. Replace one critical command with an explicit `if !` command check.

Verification: The learner can explain exactly which failures are caught by each mechanism.

### Common Mistakes and Risks

- Assuming strict mode makes a script safe.
- ERR traps that overwrite the original status.
- Printing secret-bearing commands.
- Catching errors but still returning success.

### Knowledge Check

1. What does `set -u` do?

Answer: \_\_\_\_\_

2. Why is `pipefail` important?

Answer: \_\_\_\_\_

3. Does `set -e` catch every failure?

Answer: \_\_\_\_\_

4. What should an error message contain?

Answer: \_\_\_\_\_

5. When is explicit handling preferable?

Answer: \_\_\_\_\_

### Interview Questions

139. Name contexts where `errexit` is ignored.

140. How do ERR traps propagate into functions?

141. How would you preserve the original exit status during cleanup?

### Unit Skills Checklist

- ☐ Use strict options deliberately.
- ☐ Preserve exit statuses.
- ☐ Produce useful non-secret diagnostics.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 14 - Exit Codes, Error Handling and Strict Mode completed.

# Unit 15 - Traps, Signals and Cleanup

## Why This Matters

Scripts must clean up temporary resources and respond correctly to termination, especially in CI runners and deployment processes.

## Core Concepts

| Concept                    | Explanation                                                           |
|----------------------------|-----------------------------------------------------------------------|
| <b>Signal</b>              | An asynchronous notification such as INT, TERM or HUP.                |
| <b>trap</b>                | Register shell code to run on a signal or pseudo-signal such as EXIT. |
| <b>EXIT trap</b>           | Runs when the shell exits; useful for cleanup.                        |
| <b>INT and TERM</b>        | Common interactive interruption and graceful termination signals.     |
| <b>Temporary resources</b> | Use mktemp rather than predictable filenames.                         |
| <b>Idempotent cleanup</b>  | Cleanup should tolerate partially-created resources.                  |

## Worked Examples

### Temporary directory

```
tmp_dir="$(mktemp -d)"
cleanup() {
  local rc=$?
  rm -rf -- "$tmp_dir"
  exit "$rc"
}
trap cleanup EXIT INT TERM
```

### Separate signal message

```
on_term() { printf 'Termination requested\n' >&2; exit 143; }
trap on_term TERM
```

## How Companies Use It

CI jobs receive TERM when cancelled. Deployment scripts may need to release locks, remove temporary credentials and stop child processes. Cleanup must not mask the original failure.

### Hands-on Lab: Create a safe temporary workspace

Objective: Use traps and mktemp.

Steps:

142. Create a temporary directory with `mktemp -d`.
143. Register an EXIT cleanup function.
144. Create test files inside it.
145. Force a command failure.
146. Confirm the directory is removed and the original non-zero status is retained.

Verification: No temporary directory remains and the script reports the original failure status.

## Common Mistakes and Risks

- `rm -rf` on an unvalidated variable.
- Predictable `/tmp` filenames.
- Cleanup that returns success after a failure.
- Ignoring cancellation signals.

## Knowledge Check

### 1. What is a signal?

Answer: \_\_\_\_\_

### 2. Why use an EXIT trap?

Answer: \_\_\_\_\_

### 3. Why is `mktemp` safer?

Answer: \_\_\_\_\_

### 4. How do you retain the original status?

Answer: \_\_\_\_\_

### 5. Why should cleanup be idempotent?

Answer: \_\_\_\_\_

## Interview Questions

147. What exit status commonly represents SIGTERM?

148. How do signals affect child processes?

149. What race conditions can exist around cleanup?

## Unit Skills Checklist

- ☐ Create secure temporary resources.
- ☐ Handle EXIT, INT and TERM.
- ☐ Preserve the original status.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 15 - Traps, Signals and Cleanup completed.

# Unit 16 - Logging, Timestamps and Observability

## Why This Matters

Logs are part of a script interface. Consistent structured logging accelerates troubleshooting and supports automated analysis.

## Core Concepts

| Concept              | Explanation                                                        |
|----------------------|--------------------------------------------------------------------|
| Log level            | DEBUG, INFO, WARN and ERROR convey severity.                       |
| Timestamp            | Use UTC ISO 8601 for cross-system correlation.                     |
| Context              | Include component, environment, release and operation identifiers. |
| stdout/stderr policy | Data to stdout; logs usually to stderr.                            |
| Structured log       | Key-value or JSON records are easier for machines to process.      |
| Secret hygiene       | Tokens, passwords and private keys must never appear in logs.      |

## Worked Examples

### Text logger

```
log() {  
  local level="$1"; shift  
  printf '%s level=%s message=%q\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)" "$level" "$*" >&2  
}
```

### JSON with jq

```
jq -n --arg ts "$(date -u +%Y-%m-%dT%H:%M:%SZ)" --arg level INFO --arg message "deployment started"  
'{timestamp:$ts, level:$level, message:$message}'
```

## How Companies Use It

Central logging systems ingest application and automation logs. Correlation IDs connect CI runs, deployments and incidents. Log volume and sensitivity must be controlled.

### Hands-on Lab: Add structured logging

Objective: Instrument a health-check script.

Steps:

150. Create log\_info, log\_warn and log\_error helpers.
151. Use UTC timestamps.
152. Include a run\_id generated from date and process ID.
153. Write logs to stderr.
154. Ensure a secret variable is never printed.

Verification: Logs are consistently formatted and can be separated from stdout results.

### Common Mistakes and Risks

- Inconsistent timestamps.
- Printing full environment dumps.

- Logging secret-bearing command lines.
- Using free-form logs where structured data is required.

### Knowledge Check

#### 1. Why use UTC timestamps?

Answer: \_\_\_\_\_

#### 2. What belongs in log context?

Answer: \_\_\_\_\_

#### 3. Why separate logs from data?

Answer: \_\_\_\_\_

#### 4. What is a correlation ID?

Answer: \_\_\_\_\_

#### 5. How should secrets be handled?

Answer: \_\_\_\_\_

### Interview Questions

155. How would you emit JSON safely?  
156. What is log injection?  
157. How should verbosity be controlled?

### Unit Skills Checklist

- ☐ Implement consistent logging.
- ☐ Separate data and diagnostics.
- ☐ Protect secrets.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 16 - Logging, Timestamps and Observability completed.

# Unit 17 - Files, Directories and Safe Configuration Changes

## Why This Matters

Operational scripts modify important files. Safe automation validates paths, preserves ownership and permissions, and makes changes atomically.

## Core Concepts

| Concept                  | Explanation                                                           |
|--------------------------|-----------------------------------------------------------------------|
| Atomic replacement       | Write a new file and rename it into place on the same filesystem.     |
| Backup                   | Create a controlled backup when rollback is required.                 |
| Permissions              | Use install or chmod/chown deliberately.                              |
| Path validation          | Reject empty, root or unexpected paths before destructive operations. |
| Locking                  | Prevent concurrent writers with flock.                                |
| Configuration validation | Run a syntax check before activating a new configuration.             |

## Worked Examples

### Atomic write

```
tmp="$(mktemp "${config}.XXXXXX")"
printf '%s\n' "$new_content" >"$tmp"
chmod --reference="$config" "$tmp" 2>/dev/null || chmod 0644 "$tmp"
mv -f -- "$tmp" "$config"
```

### Lock

```
exec 9>/var/lock/my-script.lock
flock -n 9 || { printf 'Already running\n' >&2; exit 1; }
```

### Validation before reload

```
nginx -t && systemctl reload nginx
```

## How Companies Use It

Configuration management systems perform declarative and repeatable changes. Bash is often used for small controlled updates, bootstrap steps and validation wrappers.

### Hands-on Lab: Safely update a configuration file

Objective: Implement backup, validation and atomic replacement.

Steps:

158. Create a sample app.conf.
159. Acquire a local lock file.
160. Write changed content to a temporary file.
161. Validate that required keys exist.
162. Atomically replace the original and retain one timestamped backup.

Verification: Concurrent execution is blocked and invalid configuration never replaces the active file.

## Common Mistakes and Risks

- Editing files in place without validation.
- Unsafe `rm -rf` paths.
- Concurrent writers.
- Changing permissions unintentionally.

## Knowledge Check

### 1. What is atomic replacement?

Answer: \_\_\_\_\_

### 2. Why validate before activation?

Answer: \_\_\_\_\_

### 3. What does flock do?

Answer: \_\_\_\_\_

### 4. Why preserve file permissions?

Answer: \_\_\_\_\_

### 5. Which paths should destructive scripts reject?

Answer: \_\_\_\_\_

## Interview Questions

163. Why must rename occur on the same filesystem for atomicity?

164. How would you implement rollback?

165. What TOCTOU risks exist?

## Unit Skills Checklist

- ☐ Make an atomic file update.
- ☐ Use flock.
- ☐ Validate and roll back safely.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 17 - Files, Directories and Safe Configuration Changes completed.

# Unit 18 - Network Automation with curl, DNS and Timeouts

## Why This Matters

Network calls are unreliable by nature. Robust scripts use explicit timeouts, status checks, retries and TLS verification.

## Core Concepts

| Concept                       | Explanation                                                        |
|-------------------------------|--------------------------------------------------------------------|
| <b>curl failure modes</b>     | DNS, connection, TLS, HTTP status, timeout and malformed response. |
| <b>Fail on HTTP errors</b>    | -f or --fail-with-body.                                            |
| <b>Silent but show errors</b> | -sS.                                                               |
| <b>Timeouts</b>               | --connect-timeout and --max-time.                                  |
| <b>Retries</b>                | Use bounded retries with delay and only for appropriate errors.    |
| <b>TLS validation</b>         | Never disable certificate verification in production.              |
| <b>DNS tools</b>              | getent hosts, dig and nslookup support diagnosis.                  |

## Worked Examples

### Safe request

```
curl --fail-with-body -sS --connect-timeout 3 --max-time 10 https://example.com/health
```

### Capture response and status

```
http_code="$(curl -sS -o response.json -w '%{http_code}' --max-time 10 "$url")"
```

### DNS check

```
getent ahosts example.com
```

## How Companies Use It

Deployment verification and synthetic monitoring use network checks. Production systems should use dedicated monitoring for continuous availability, while scripts handle gated checks and diagnostics.

### Hands-on Lab: Create an API readiness check

Objective: Validate an HTTP endpoint reliably.

Steps:

166. Accept a URL and expected status code.
167. Use connection and total timeouts.
168. Retry at most five times with increasing delay.
169. Record the final HTTP status and total time.
170. Return success only when the expected status is received.

Verification: An unreachable endpoint fails within a bounded time; a valid endpoint succeeds.

### Common Mistakes and Risks

- No timeout.
- Infinite retries.

- Disabling TLS verification.
- Retrying non-idempotent requests blindly.

### Knowledge Check

#### 1. Why are timeouts mandatory?

Answer: \_\_\_\_\_

#### 2. What does curl -f do?

Answer: \_\_\_\_\_

#### 3. Why avoid -k?

Answer: \_\_\_\_\_

#### 4. What should be retried?

Answer: \_\_\_\_\_

#### 5. How do DNS and HTTP failures differ?

Answer: \_\_\_\_\_

### Interview Questions

171. How do you distinguish transport failure from HTTP error?
172. What is exponential backoff with jitter?
173. Why can retries amplify outages?

### Unit Skills Checklist

- ☐ Use curl safely.
- ☐ Implement bounded retries.
- ☐ Diagnose DNS versus transport versus HTTP.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 18 - Network Automation with curl, DNS and Timeouts completed.

# Unit 19 - JSON, YAML and APIs with jq and yq

## Why This Matters

Modern DevOps tools exchange structured data. Use structured parsers instead of `grep` and `sed`.

## Core Concepts

| Concept           | Explanation                                                                              |
|-------------------|------------------------------------------------------------------------------------------|
| JSON              | A structured data format with objects, arrays, strings, numbers, booleans and null.      |
| jq                | Query, transform and construct JSON.                                                     |
| YAML              | Human-friendly structured format used by Kubernetes, CI systems and configuration tools. |
| yq                | A YAML processor; implementations differ, so verify the installed version.               |
| API pagination    | Large result sets may require multiple requests.                                         |
| Schema validation | Validate required fields and types before acting.                                        |

## Worked Examples

### Read JSON

```
jq -r '.status' response.json
```

### Filter array

```
jq -r '.items[] | select(.ready == false) | .name' response.json
```

### Construct JSON

```
jq -n --arg service api --arg version v1.2.3 '{service:$service,version:$version}'
```

### YAML query

```
yq -r '.metadata.name' deployment.yaml
```

## How Companies Use It

Cloud CLIs and Kubernetes tools provide JSON output for automation. Production scripts pin compatible tool versions and validate response fields.

### Hands-on Lab: Process an API response

Objective: Extract readiness and fail safely.

Steps:

174. Create a JSON file with a list of services and readiness states.
175. Use `jq` to list unready services.
176. Count unready services.
177. Reject malformed or missing data.
178. Return non-zero when any service is unready.

Verification: Structured parsing works regardless of whitespace or field order.

### Common Mistakes and Risks

- String-concatenating JSON manually.

- Assuming every API response is successful JSON.
- Ignoring pagination.
- Depending on an unspecified yq implementation.

### Knowledge Check

#### 1. Why not parse JSON with grep?

Answer: \_\_\_\_\_

#### 2. What does jq -r do?

Answer: \_\_\_\_\_

#### 3. How do you construct JSON safely?

Answer: \_\_\_\_\_

#### 4. Why verify yq implementation and version?

Answer: \_\_\_\_\_

#### 5. What is pagination?

Answer: \_\_\_\_\_

### Interview Questions

179. How do you make jq fail when a field is absent?

180. How would you follow paginated API links?

181. How can schema validation improve safety?

### Unit Skills Checklist

- ☐ Query JSON with jq.
- ☐ Construct JSON safely.
- ☐ Validate structured responses.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 19 - JSON, YAML and APIs with jq and yq completed.

# Unit 20 - Scheduling with cron and systemd Timers

## Why This Matters

Automation must run reliably at the correct time, with a controlled environment and observable results.

## Core Concepts

| Concept                   | Explanation                                                       |
|---------------------------|-------------------------------------------------------------------|
| <b>cron</b>               | Traditional time-based scheduler.                                 |
| <b>Crontab fields</b>     | Minute, hour, day of month, month and day of week.                |
| <b>Environment</b>        | Cron provides a limited environment and PATH.                     |
| <b>systemd timer</b>      | Pairs a .timer with a .service for richer scheduling and logging. |
| <b>Persistent timer</b>   | Can run missed jobs after downtime.                               |
| <b>Overlap protection</b> | Use flock or service controls to avoid concurrent runs.           |

## Worked Examples

### Cron entry

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
0 2 * * * /opt/scripts/backup.sh >>/var/log/backup.log 2>&1
```

### Timer

```
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
Unit=backup.service
```

### Service

```
[Service]
Type=oneshot
ExecStart=/opt/scripts/backup.sh
```

## How Companies Use It

Companies prefer schedulers that expose history, logs, retries and ownership. cron remains common for simple host-local jobs; Kubernetes CronJobs and workflow platforms serve containerised environments.

### Hands-on Lab: Schedule a safe local job

Objective: Create a recurring script with logging.

Steps:

182. Create a script that writes a timestamp and disk usage to a log.
183. Run it manually with an explicit PATH.
184. Create either a user cron entry or a user systemd timer.
185. Prevent overlap with flock.
186. Verify the scheduler history and logs.

Verification: At least two scheduled executions are visible and overlapping execution is prevented.

## Common Mistakes and Risks

- Assuming interactive environment variables exist.
- No logging.
- Overlapping jobs.
- Scheduling privileged scripts without least privilege.

## Knowledge Check

### 1. Why does cron need an explicit PATH?

Answer: \_\_\_\_\_

### 2. What is a persistent timer?

Answer: \_\_\_\_\_

### 3. How do you inspect timer logs?

Answer: \_\_\_\_\_

### 4. Why prevent overlap?

Answer: \_\_\_\_\_

### 5. When is Kubernetes CronJob more appropriate?

Answer: \_\_\_\_\_

## Interview Questions

187. What does cron day-of-month plus day-of-week semantics mean?

188. How do systemd timers randomise load?

189. How do you handle missed schedules?

## Unit Skills Checklist

- ☐ Create a scheduled job.
- ☐ Provide explicit environment and logs.
- ☐ Prevent concurrency.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 20 - Scheduling with cron and systemd Timers completed.

# Unit 21 - Remote Execution, SSH and Fleet Safety

## Why This Matters

Remote automation can affect many systems quickly. It requires strong host verification, bounded concurrency and careful failure reporting.

## Core Concepts

| Concept               | Explanation                                                          |
|-----------------------|----------------------------------------------------------------------|
| SSH keys              | Preferred over passwords for automation.                             |
| Host key verification | Prevents connecting to an impersonated server.                       |
| BatchMode             | Disable interactive password prompts.                                |
| ConnectTimeout        | Bound connection establishment time.                                 |
| Remote quoting        | Local and remote shells may both interpret characters.               |
| Concurrency           | Parallel execution reduces time but increases load and blast radius. |
| Inventory             | Maintain explicit host lists and environment boundaries.             |

## Worked Examples

### Safe non-interactive SSH

```
ssh -o BatchMode=yes -o ConnectTimeout=5 user@host 'hostname; uptime'
```

### Copy file

```
scp -p deploy.sh user@host:/tmp/deploy.sh
```

### Loop with results

```
for host in "${hosts[@]"; do
  if ssh -o BatchMode=yes "$host" 'systemctl is-active --quiet nginx'; then
    printf '%s OK\n' "$host"
  else
    printf '%s FAIL\n' "$host" >&2
  fi
done
```

## How Companies Use It

At scale, companies use Ansible, orchestration platforms, cloud systems management, Kubernetes or immutable deployments rather than unmanaged SSH loops.

### Hands-on Lab: Create a local SSH readiness plan

Objective: Design without touching production systems.

Steps:

190. Create an inventory file with dev host aliases only.
191. Validate that each hostname matches an approved suffix.
192. Print the exact SSH command in dry-run mode.
193. Require an explicit `--execute` option before connecting.
194. Summarise success and failure per host.

Verification: The default mode makes no remote changes and unapproved hosts are rejected.

### Common Mistakes and Risks

- StrictHostKeyChecking=no in production.
- Unlimited parallel SSH.
- Mixing production and development inventory.
- No dry-run or confirmation boundary.

### Knowledge Check

#### 1. Why is host key verification important?

Answer: \_\_\_\_\_

#### 2. What does BatchMode do?

Answer: \_\_\_\_\_

#### 3. Why is remote quoting difficult?

Answer: \_\_\_\_\_

#### 4. What is blast radius?

Answer: \_\_\_\_\_

#### 5. When should Ansible replace SSH loops?

Answer: \_\_\_\_\_

### Interview Questions

195. How would you cap parallelism?

196. How do jump hosts affect SSH configuration?

197. How can ControlMaster improve performance and create risk?

### Unit Skills Checklist

- ☐ Use safe SSH options.
- ☐ Validate inventories.
- ☐ Control blast radius.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 21 - Remote Execution, SSH and Fleet Safety completed.

# Unit 22 - Packaging, Portability and Dependencies

## Why This Matters

A script that works only on one laptop is not production-ready. Portability requires dependency checks and documented assumptions.

## Core Concepts

| Concept                      | Explanation                                                                             |
|------------------------------|-----------------------------------------------------------------------------------------|
| <b>Bash version</b>          | Features differ across Bash releases.                                                   |
| <b>GNU versus BSD tools</b>  | Options for sed, date, stat and others differ.                                          |
| <b>Dependency check</b>      | Use command -v and version validation.                                                  |
| <b>Self-contained layout</b> | Separate bin, lib, config, tests and documentation.                                     |
| <b>PATH independence</b>     | Resolve script directory rather than assuming the current directory.                    |
| <b>Packaging</b>             | Distribute scripts through repositories, packages, container images or artefact stores. |

## Worked Examples

### Script directory

```
SCRIPT_DIR="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)"
```

### Dependency check

```
for cmd in curl jq; do
  command -v "$cmd" >/dev/null 2>&1 || { printf 'Missing: %s\n' "$cmd" >&2; exit 1; }
done
```

### Version guard

```
if (( BASH_VERSION[0] < 4 )); then
  printf 'Bash 4 or newer required\n' >&2
  exit 1
fi
```

## How Companies Use It

Internal platform teams publish versioned command-line tools and containerised utilities. Scripts are tested on supported operating systems and pinned dependency versions.

### Hands-on Lab: Package a script project

Objective: Create a clean repository layout.

Steps:

198. Create bin, lib, config, tests and docs directories.
199. Move common functions into lib/common.sh.
200. Resolve all paths relative to the script location.
201. Add dependency and Bash-version checks.
202. Write a README with supported platforms and usage.

Verification: The tool works when launched from a different current directory.

## Common Mistakes and Risks

- Using relative paths against pwd.
- Undocumented command dependencies.
- Assuming GNU options everywhere.
- Copying mutable scripts manually between servers.

## Knowledge Check

### 1. Why is current directory unreliable?

Answer: \_\_\_\_\_

### 2. What differs between GNU and BSD tools?

Answer: \_\_\_\_\_

### 3. How do you check Bash version?

Answer: \_\_\_\_\_

### 4. Why document dependencies?

Answer: \_\_\_\_\_

### 5. When is a container a useful delivery mechanism?

Answer: \_\_\_\_\_

## Interview Questions

203. How can you avoid sourcing path vulnerabilities?

204. What is semantic versioning for scripts?

205. How do reproducible builds apply to utility images?

## Unit Skills Checklist

- ☐ Resolve script paths correctly.
- ☐ Check versions and dependencies.
- ☐ Package a maintainable script project.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 22 - Packaging, Portability and Dependencies completed.

# Unit 23 - Security for Shell Scripts

## Why This Matters

Shell scripts often handle privileged operations, secrets and user input. Small mistakes can become command injection or data-loss vulnerabilities.

## Core Concepts

| Concept                  | Explanation                                                                       |
|--------------------------|-----------------------------------------------------------------------------------|
| <b>Least privilege</b>   | Run with only the permissions required.                                           |
| <b>Command injection</b> | Untrusted text becomes executable syntax.                                         |
| <b>Input validation</b>  | Prefer strict allowlists and reject unexpected data.                              |
| <b>Temporary files</b>   | Use mktemp and restrictive permissions.                                           |
| <b>Secrets</b>           | Use secret managers, protected files or injected environment variables with care. |
| <b>PATH security</b>     | Use a controlled PATH and absolute commands for privileged scripts.               |
| <b>ShellCheck</b>        | Static analysis catches many quoting and safety defects.                          |

## Worked Examples

### Unsafe

```
eval "command $user_input"
```

### Safer argument array

```
cmd=(curl --fail-with-body -sS)
cmd+=("--header "Authorization: Bearer $token")
cmd+=("$url")
"${cmd[@]}"
```

### Controlled PATH

```
PATH=/usr/sbin:/usr/bin:/sbin:/bin
export PATH
```

### Sensitive file

```
umask 077
secret_file="$(mktemp)"
```

## How Companies Use It

Security reviews inspect privilege boundaries, input sources, secret exposure, dependency provenance and destructive operations. ShellCheck commonly runs in CI.

### Hands-on Lab: Harden an input-driven script

Objective: Remove injection and secret leakage risks.

Steps:

206. Create a deliberately unsafe script that builds a command string from input, but do not execute malicious input.
207. Replace the string with an argument array.
208. Add an allowlist for environment names.
209. Set a restrictive umask for temporary secrets.

210. Run shellcheck and address relevant findings.

Verification: The final script contains no eval, quotes data safely and passes agreed ShellCheck rules.

### Common Mistakes and Risks

- eval with untrusted input.
- World-readable temporary files.
- Passing secrets on command lines visible to process listings.
- Running the entire script as root.

### Knowledge Check

1. Why is eval dangerous?

Answer: \_\_\_\_\_

2. What is least privilege?

Answer: \_\_\_\_\_

3. Why use argument arrays?

Answer: \_\_\_\_\_

4. How should temporary secret files be protected?

Answer: \_\_\_\_\_

5. What does ShellCheck provide?

Answer: \_\_\_\_\_

### Interview Questions

211. How can environment variables leak?

212. What is PATH hijacking?

213. How would you threat-model a deployment script?

### Unit Skills Checklist

- ☐ Avoid command injection.
- ☐ Protect secrets and temporary data.
- ☐ Run static analysis.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 23 - Security for Shell Scripts completed.

# Unit 24 - Testing Bash with ShellCheck, shfmt and Bats

## Why This Matters

Testing turns scripts from personal helpers into maintainable engineering artefacts.

## Core Concepts

| Concept              | Explanation                                            |
|----------------------|--------------------------------------------------------|
| Static analysis      | ShellCheck detects common bugs without executing code. |
| Formatting           | shfmt enforces consistent layout.                      |
| Unit test            | Test a small function or behaviour in isolation.       |
| Integration test     | Exercise interactions with files or commands.          |
| Bats                 | Bash Automated Testing System.                         |
| Test doubles         | Stub or fake external commands.                        |
| Exit-code assertions | Validate both output and status.                       |

## Worked Examples

### ShellCheck

```
shellcheck bin/deploy.sh
```

### shfmt

```
shfmt -d .
shfmt -w .
```

### Bats example

```
@test "rejects invalid environment" {
  run ./bin/deploy.sh -e invalid -v v1.0.0
  [ "$status" -eq 2 ]
  [[ "$output" == *"Invalid environment"* ]]
}
```

## How Companies Use It

CI pipelines run formatting, static analysis and tests on every pull request. Critical operational tools may also run tests inside multiple container images to verify compatibility.

### Hands-on Lab: Test the deployment parser

Objective: Create automated behaviour checks.

Steps:

214. Install or use a container providing ShellCheck, shfmt and Bats.
215. Write tests for valid dev and prod options.
216. Write tests for missing arguments and invalid environment.
217. Add a fake external command to test a failure path.
218. Run all checks with one make or shell command.

Verification: Formatting, static analysis and all tests pass; intentionally broken behaviour produces a failing test.

## Common Mistakes and Risks

- Only testing the happy path.
- Assertions that ignore exit status.
- Tests that contact real production services.
- Treating ShellCheck as infallible.

## Knowledge Check

### 1. What is static analysis?

Answer: \_\_\_\_\_

### 2. Why test exit codes?

Answer: \_\_\_\_\_

### 3. What is an integration test?

Answer: \_\_\_\_\_

### 4. How can external commands be faked?

Answer: \_\_\_\_\_

### 5. Why run checks in CI?

Answer: \_\_\_\_\_

## Interview Questions

219. How do you test signal handling?

220. What is mutation testing conceptually?

221. How can golden-file tests become brittle?

## Unit Skills Checklist

- ☐ Run ShellCheck and shfmt.
- ☐ Write Bats tests.
- ☐ Test success and failure behaviour.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 24 - Testing Bash with ShellCheck, shfmt and Bats completed.

# Unit 25 - Debugging and Performance Analysis

## Why This Matters

When scripts fail, disciplined debugging is faster and safer than adding random echo statements.

## Core Concepts

| Concept                   | Explanation                                                       |
|---------------------------|-------------------------------------------------------------------|
| <b>Syntax check</b>       | bash -n.                                                          |
| <b>Execution trace</b>    | bash -x or set -x; beware secret leakage.                         |
| <b>Custom PS4</b>         | Include source, line and function in traces.                      |
| <b>Time measurement</b>   | Use time, date or hyperfine where available.                      |
| <b>Process inspection</b> | ps, pstree, lsof and strace support diagnosis.                    |
| <b>Subshell cost</b>      | External commands and command substitutions add process overhead. |
| <b>Profiling mindset</b>  | Measure before optimising.                                        |

## Worked Examples

### Detailed trace

```
export PS4='+${BASH_SOURCE}:${LINENO}:${FUNCNAME[0]}: '
BASH_XTRACEFD=9
exec 9>trace.log
set -x
```

### Timing

```
start_ns="$(date +%s%N)"
# work
end_ns="$(date +%s%N)"
printf 'elapsed_ns=%s\n' "$((end_ns-start_ns))"
```

## How Companies Use It

Incident responders reproduce failures in safe environments, capture exact inputs and compare known-good versions. Performance problems may indicate that a higher-level language or parallel tool is more appropriate.

### Hands-on Lab: Debug a failing script

Objective: Use a repeatable diagnostic workflow.

Steps:

222. Create a script with an unset variable, a failed command and a quoting bug.
223. Run bash -n first.
224. Run with a trace redirected to a protected file.
225. Identify the earliest incorrect assumption.
226. Add a regression test after fixing each defect.

Verification: All defects are explained, fixed and covered by tests without exposing secrets in traces.

### Common Mistakes and Risks

- Tracing secrets.
- Fixing symptoms instead of root causes.

- Optimising unmeasured code.
- Using many external processes inside tight loops.

### Knowledge Check

#### 1. What does bash -n detect?

Answer: \_\_\_\_\_

#### 2. Why can set -x be dangerous?

Answer: \_\_\_\_\_

#### 3. What does PS4 control?

Answer: \_\_\_\_\_

#### 4. Why measure before optimising?

Answer: \_\_\_\_\_

#### 5. When does Bash become the wrong performance tool?

Answer: \_\_\_\_\_

### Interview Questions

227. How does BASH\_XTRACEFD help?

228. When is strace useful?

229. Why are forks expensive in large loops?

### Unit Skills Checklist

- ☐ Use a structured debugging workflow.
- ☐ Capture safe traces.
- ☐ Add regression tests.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 25 - Debugging and Performance Analysis completed.

# Unit 26 - Backup, Retention and Recovery Automation

## Why This Matters

A backup is useful only when it can be restored. Automation must verify results, enforce retention safely and surface failures.

## Core Concepts

| Concept             | Explanation                                                                            |
|---------------------|----------------------------------------------------------------------------------------|
| <b>Backup scope</b> | Identify data, configuration and metadata required for recovery.                       |
| <b>Archive</b>      | tar groups files; compression is optional.                                             |
| <b>Integrity</b>    | Checksums detect accidental corruption but do not prove authenticity unless protected. |
| <b>Retention</b>    | Delete only files matching an exact controlled policy.                                 |
| <b>Remote copy</b>  | Use rsync or object storage with secure credentials.                                   |
| <b>Restore test</b> | Regularly prove that recovery works.                                                   |
| <b>RPO and RTO</b>  | Recovery point and recovery time objectives.                                           |

## Worked Examples

### Archive

```
tar -C /srv -czf "/backup/app-$(date -u +%Y%m%dT%H%M%SZ).tar.gz" app
```

### Checksum

```
sha256sum backup.tar.gz > backup.tar.gz.sha256  
sha256sum -c backup.tar.gz.sha256
```

### Retention preview

```
find /backup -maxdepth 1 -type f -name 'app-*.tar.gz' -mtime +30 -print
```

### Rsync

```
rsync -aH --delete-delay --dry-run /srv/app/ backup-host:/backups/app/
```

## How Companies Use It

Companies combine application-aware backups, database-native tools, snapshots and object-storage lifecycle policies. Restore drills are part of disaster recovery.

### Hands-on Lab: Create a verifiable local backup

Objective: Back up sample application data safely.

Steps:

230. Create sample data and configuration.
231. Generate a timestamped archive in a dedicated backup directory.
232. Generate and verify a SHA-256 checksum.
233. List retention candidates older than a chosen threshold before deletion.
234. Restore into a separate directory and compare contents.

Verification: Checksum passes and restored data matches the original.

## Common Mistakes and Risks

- Deleting with broad find patterns.
- No restore testing.
- Backing up live databases with naive file copies.
- Storing backup credentials in scripts.

## Knowledge Check

### 1. What is the difference between backup and archive?

Answer: \_\_\_\_\_

### 2. Why are restore tests necessary?

Answer: \_\_\_\_\_

### 3. What are RPO and RTO?

Answer: \_\_\_\_\_

### 4. Why preview retention deletion?

Answer: \_\_\_\_\_

### 5. What does rsync --dry-run provide?

Answer: \_\_\_\_\_

## Interview Questions

235. How do database-consistent backups differ from file copies?

236. How should encryption keys be managed?

237. What is an immutable backup?

## Unit Skills Checklist

- ☐ Create and verify a backup.
- ☐ Implement safe retention.
- ☐ Perform a restore test.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 26 - Backup, Retention and Recovery Automation completed.

# Unit 27 - Deployment Automation Patterns

## Why This Matters

Deployment scripts must be repeatable, observable and reversible. They should reduce risk rather than merely automate commands.

## Core Concepts

| Concept                      | Explanation                                                                    |
|------------------------------|--------------------------------------------------------------------------------|
| <b>Pre-flight</b>            | Validate dependencies, credentials, environment and artefact.                  |
| <b>Artefact immutability</b> | Deploy a versioned, checksummed artefact rather than rebuilding on the server. |
| <b>Maintenance strategy</b>  | Rolling, blue-green and canary approaches manage risk differently.             |
| <b>Health verification</b>   | Check readiness and functional behaviour after change.                         |
| <b>Rollback</b>              | Return to a known-good version using a tested procedure.                       |
| <b>Dry-run</b>               | Show planned actions without changing state.                                   |
| <b>Audit trail</b>           | Record who, what, where, when and result.                                      |

## Worked Examples

### Deployment phases

```
validate
download_artifact
verify_checksum
install_release
switch_symlink
restart_service
verify_health
```

### Versioned release layout

```
/opt/myapp/releases/v1.4.2
/opt/myapp/current -> /opt/myapp/releases/v1.4.2
```

### Rollback

```
ln -sfn "/opt/myapp/releases/$previous" /opt/myapp/current
systemctl restart myapp
```

## How Companies Use It

Companies use CI/CD platforms and deployment controllers, but shell scripts often implement hooks, validation and host-local release operations. Production changes use approvals and environment protection.

### Hands-on Lab: Simulate an atomic release deployment

Objective: Deploy a static application into versioned directories.

Steps:

238. Create releases/v1 and releases/v2 directories with distinct content.

239. Create a current symlink pointing to v1.

240. Write a deployment script that validates v2, atomically switches current and runs a health check.

241. Force the health check to fail and automatically restore v1.

242. Log the release, result and rollback.

Verification: Successful deployments remain on v2; failed deployments return to v1.

### Common Mistakes and Risks

- Building artefacts on production servers.
- No health verification.
- Rollback code never tested.
- Deploying mutable latest tags.

### Knowledge Check

#### 1. What is an immutable artefact?

Answer: \_\_\_\_\_

#### 2. Why separate pre-flight and execution?

Answer: \_\_\_\_\_

#### 3. What is blue-green deployment?

Answer: \_\_\_\_\_

#### 4. What must rollback restore?

Answer: \_\_\_\_\_

#### 5. Why use a dry-run?

Answer: \_\_\_\_\_

### Interview Questions

243. How do database migrations complicate rollback?

244. What is a deployment transaction?

245. How should progressive delivery metrics influence automation?

### Unit Skills Checklist

- ☐ Implement phased deployment logic.
- ☐ Verify and roll back.
- ☐ Create an audit trail.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 27 - Deployment Automation Patterns completed.

# Unit 28 - CI/CD Integration for Shell Scripts

## Why This Matters

Shell scripts often become the reusable implementation behind CI/CD pipeline steps. They must work non-interactively and return trustworthy statuses.

## Core Concepts

| Concept           | Explanation                                                        |
|-------------------|--------------------------------------------------------------------|
| CI environment    | Ephemeral worker with injected variables and limited credentials.  |
| Non-interactive   | No prompts unless explicitly supported by the platform.            |
| Artefacts         | Files produced for later stages or download.                       |
| Cache             | Reusable dependency data; not a source of truth.                   |
| Secret masking    | Platform feature that reduces but does not eliminate leakage risk. |
| Pipeline contract | Inputs, outputs and statuses must be documented.                   |
| Matrix test       | Run across versions or platforms.                                  |

## Worked Examples

### CI entrypoint

```
#!/usr/bin/env bash
set -Eeuo pipefail
: "${CI_COMMIT_SHA:?required}"
./scripts/lint.sh
./scripts/test.sh
```

### GitHub Actions step

```
- name: Validate shell scripts
  run: |
    shellcheck scripts/*.sh
    bats tests
```

### GitLab job

```
shell_quality:
  image: koalaman/shellcheck-alpine
  script:
    - shellcheck scripts/*.sh
```

## How Companies Use It

Teams keep pipeline YAML thin and move reusable logic into versioned scripts or actions. Changes to shared scripts receive code review and automated tests.

### Hands-on Lab: Create a CI-ready quality command

Objective: Make one script execute all quality gates.

Steps:

246. Create scripts/ci.sh.

247. Ensure it never prompts.

248. Check required commands.

249. Run shfmt check, ShellCheck and Bats.

250. Return the first meaningful non-zero status and print a concise summary.

Verification: The command succeeds locally and behaves correctly in a clean environment or container.

### Common Mistakes and Risks

- Prompts in CI.
- Printing all environment variables.
- Pipeline YAML duplicating complex logic.
- Ignoring tool-version drift.

### Knowledge Check

#### 1. Why should CI scripts be non-interactive?

Answer: \_\_\_\_\_

#### 2. What is an artefact?

Answer: \_\_\_\_\_

#### 3. Why keep pipeline YAML thin?

Answer: \_\_\_\_\_

#### 4. How are secrets injected?

Answer: \_\_\_\_\_

#### 5. What should a quality gate return?

Answer: \_\_\_\_\_

### Interview Questions

251. How do caches become poisoned or stale?

252. What is a matrix build?

253. How do reusable workflow interfaces evolve safely?

### Unit Skills Checklist

- ☐ Build a CI entry point.
- ☐ Return trustworthy statuses.
- ☐ Protect secrets and outputs.

### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 28 - CI/CD Integration for Shell Scripts completed.

# Unit 29 - Capstone Project: Production Health and Readiness Toolkit

## Why This Matters

This project integrates CLI design, structured logging, networking, JSON parsing, retries, tests and packaging.

## Core Concepts

| Concept     | Explanation                                                                                                             |
|-------------|-------------------------------------------------------------------------------------------------------------------------|
| Goal        | Create a reusable toolkit that checks DNS, TCP/HTTP reachability, expected status, response JSON and service readiness. |
| Inputs      | Configuration file plus command-line overrides.                                                                         |
| Outputs     | Human summary and optional JSON report.                                                                                 |
| Reliability | Timeouts, bounded retries and deterministic exit codes.                                                                 |
| Safety      | No secrets in logs; TLS verification enabled.                                                                           |
| Quality     | ShellCheck, shfmt and Bats tests.                                                                                       |

## Worked Examples

### Suggested layout

```
health-toolkit/  
  bin/health-check  
  lib/common.sh  
  config/services.json  
  tests/health-check.bats  
  docs/README.md
```

### Exit code design

```
0 = all healthy  
1 = one or more services unhealthy  
2 = usage or configuration error  
3 = missing dependency
```

## How Companies Use It

Comparable tools run as deployment gates, synthetic probes or incident diagnostics. Continuous monitoring should still be handled by Prometheus, cloud monitoring or dedicated platforms.

### Hands-on Lab: Build the full toolkit

Objective: Implement and test a production-style health utility.

Steps:

254. Design the CLI and exit-code contract.
255. Read service definitions from JSON using jq.
256. Check each endpoint with timeout and bounded retry.
257. Produce a human summary and JSON report.
258. Add tests for success, HTTP failure, timeout and malformed configuration.
259. Run ShellCheck and shfmt.
260. Write operational documentation and examples.

Verification: All tests pass, reports are valid, failures are bounded, and the README explains installation and use.

### Common Mistakes and Risks

- No stable exit-code contract.
- Tests dependent on public internet.
- Mixing logs into JSON output.
- Unbounded execution time.

### Knowledge Check

#### 1. What are the toolkit inputs and outputs?

Answer: \_\_\_\_\_

#### 2. How are failure classes represented?

Answer: \_\_\_\_\_

#### 3. Why provide JSON and human formats?

Answer: \_\_\_\_\_

#### 4. How are retries bounded?

Answer: \_\_\_\_\_

#### 5. How is the tool tested without relying on production services?

Answer: \_\_\_\_\_

### Interview Questions

261. How would you add concurrency safely?
262. How would you publish and version the toolkit?
263. How would you integrate it into Kubernetes readiness verification?

### Unit Skills Checklist

- ☐ Complete the repository.
- ☐ Pass all quality gates.
- ☐ Demonstrate at least four failure scenarios.

#### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 29 - Capstone Project: Production Health and Readiness Toolkit completed.

# Unit 30 - Capstone Project: Backup and Restore Orchestrator

## Why This Matters

This project integrates filesystem safety, locking, checksums, retention, remote transfer, logging and recovery testing.

## Core Concepts

| Concept             | Explanation                                                            |
|---------------------|------------------------------------------------------------------------|
| <b>Goal</b>         | Create an orchestrator for application data and configuration backups. |
| <b>Manifest</b>     | Record files, sizes, checksums, timestamp and tool version.            |
| <b>Locking</b>      | Prevent overlapping backup runs.                                       |
| <b>Retention</b>    | Support dry-run and exact controlled selection.                        |
| <b>Verification</b> | Validate archive and checksum before declaring success.                |
| <b>Restore</b>      | Restore to a separate target and validate required files.              |

## Worked Examples

### Suggested phases

```
preflight -> lock -> snapshot/archive -> checksum -> manifest -> remote copy -> retention -> verify
-> report
```

### Audit record

```
run_id, source, destination, archive, checksum, duration, status
```

## How Companies Use It

This resembles host-level backup wrappers. Enterprise backup requires application-consistent snapshots, encryption, central policy and monitored restore drills.

### Hands-on Lab: Build the orchestrator

Objective: Create backup and restore subcommands.

Steps:

- 264. Design backup, verify, list and restore subcommands.
- 265. Use a lock and secure temporary directory.
- 266. Create a timestamped archive and checksum.
- 267. Create a JSON manifest with jq.
- 268. Implement retention preview and explicit execution.
- 269. Restore into a selected empty directory.
- 270. Add tests for missing source, corruption and retention boundaries.

Verification: A corrupted archive fails verification; a valid archive restores and matches the source.

### Common Mistakes and Risks

- Treating archive creation as proof of backup.
- Retention deleting unverified newest backups.
- Restore overwriting the source.

- Missing encryption and access controls.

### Knowledge Check

**1. Why is a manifest valuable?**

Answer: \_\_\_\_\_

**2. What does the lock protect?**

Answer: \_\_\_\_\_

**3. Why separate retention preview and execution?**

Answer: \_\_\_\_\_

**4. How do you verify restoration?**

Answer: \_\_\_\_\_

**5. Which failures must alert an operator?**

Answer: \_\_\_\_\_

### Interview Questions

271.How would you integrate object storage?

272.How do immutable backups resist ransomware?

273.How would you meet an RPO of 15 minutes?

### Unit Skills Checklist

- ☐ Complete backup, verify and restore commands.
- ☐ Test corruption handling.
- ☐ Document recovery steps.

#### Completion Gate

Mark this unit complete only after you can explain the concepts without notes, reproduce the lab, and answer at least 80% of the knowledge-check questions.

Status: ☐ Not started ☐ In progress ☐ Completed

Completion message format: Volume 2, Unit 30 - Capstone Project: Backup and Restore Orchestrator completed.

# Final Interview Preparation Bank

## Beginner Questions

- 274. What is Bash, and how is it different from the Linux kernel?
- 275. Explain the shebang line.
- 276. What is the difference between a shell variable and an environment variable?
- 277. Why should variables normally be quoted?
- 278. Explain `$#`, `$@`, `$?` and `$$`.
- 279. What is the difference between `[ ]` and `[[ ]]`?
- 280. How do you check whether a command exists?
- 281. What are `stdout` and `stderr`?
- 282. What is a pipeline?
- 283. How do you read a file safely line by line?

## Intermediate Questions

- 284. Explain `set -Euo pipefail` and its limitations.
- 285. What is the difference between executing and sourcing a script?
- 286. How do traps preserve cleanup and exit status?
- 287. How do you write an atomic configuration update?
- 288. Why is `for` item in `$(cat file)` unsafe?
- 289. How do you safely construct a command with dynamic arguments?
- 290. How do you validate and process JSON?
- 291. How would you implement bounded retry with backoff?
- 292. How do you prevent overlapping scheduled jobs?
- 293. How do you test a script that invokes `curl` or `systemctl`?

## Advanced and Scenario Questions

- 294. A deployment script succeeded but the application is unavailable. Design a safer deployment flow.
- 295. A cron job works manually but fails when scheduled. Diagnose it.
- 296. A script intermittently deletes the wrong backup files. How do you investigate and redesign retention?
- 297. A CI trace exposed a token while `set -x` was enabled. What immediate and preventive actions are required?
- 298. A pipeline `grep | head` returns a surprising non-zero status under `pipefail`. Explain likely causes.
- 299. A script must update 1,000 servers. Why is a Bash SSH loop risky, and what architecture would you choose?
- 300. How would you design stable exit-code contracts for a shared internal tool?
- 301. How do database migrations affect deployment rollback?
- 302. What controls reduce command-injection risk?
- 303. When should a Bash tool be rewritten in Python or Go?

# Final Practical Examination

Complete this examination in a clean laboratory without referring to completed solutions. Keep all evidence in a Git repository.

| Task                                 | Requirement                                                                                                      | Status                                                           |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>Task 1 - CLI and Validation</b>   | Create a tool accepting environment, version and dry-run options. Reject invalid values and document exit codes. | <input type="checkbox"/> Pending <input type="checkbox"/> Passed |
| <b>Task 2 - Health Check</b>         | Check three endpoints with timeouts and bounded retries. Produce human and JSON output.                          | <input type="checkbox"/> Pending <input type="checkbox"/> Passed |
| <b>Task 3 - Configuration Safety</b> | Atomically update a configuration file under a lock and validate it before activation.                           | <input type="checkbox"/> Pending <input type="checkbox"/> Passed |
| <b>Task 4 - Logging</b>              | Emit UTC structured logs with a run ID while keeping stdout machine-readable.                                    | <input type="checkbox"/> Pending <input type="checkbox"/> Passed |
| <b>Task 5 - Error Handling</b>       | Preserve original errors through traps and cleanup.                                                              | <input type="checkbox"/> Pending <input type="checkbox"/> Passed |
| <b>Task 6 - Testing</b>              | Add ShellCheck, shfmt and at least six Bats tests including failure paths.                                       | <input type="checkbox"/> Pending <input type="checkbox"/> Passed |
| <b>Task 7 - Scheduling</b>           | Create a systemd timer or cron schedule with explicit environment, logs and overlap prevention.                  | <input type="checkbox"/> Pending <input type="checkbox"/> Passed |
| <b>Task 8 - Security Review</b>      | Identify and remediate injection, PATH, temporary-file and secret-exposure risks.                                | <input type="checkbox"/> Pending <input type="checkbox"/> Passed |

## Scoring Rubric

- Correctness - 30 points
- Safety and error handling - 20 points
- Readability and modularity - 15 points
- Testing - 15 points
- Observability - 10 points
- Documentation - 10 points

### Passing Standard

Minimum 80/100 overall, no critical security defect, all destructive operations protected by validation or dry-run, and all tests reproducible.

# Bash Command and Syntax Reference

| Area            | Reference                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------|
| Execution       | bash script.sh   ./script.sh   source script.sh   bash -n   bash -x                                  |
| Variables       | name="value"   export NAME=value   readonly NAME   \${NAME:-default}   \${NAME:?message}             |
| Arguments       | \$0 \$1 \$2 \$# "\$@" \$? \$\$   shift   getopt                                                      |
| Conditions      | [[ -f file ]]   [[ -n string ]]   (( n > 5 ))   command -v tool                                      |
| Loops           | for x in "\${array[@]"; do ...; done   while condition; do ...; done   until condition; do ...; done |
| Functions       | name() { local value="\$1"; ...; return 0; }                                                         |
| Redirection     | > >> < 2> 2>&1 &>   tee   <<EOF   <<<                                                                |
| Safety          | set -Eeuo pipefail   trap cleanup EXIT INT TERM   mktemp   flock                                     |
| Quality         | shellcheck script.sh   shfmt -d .   bats tests                                                       |
| Networking      | curl -fsS --connect-timeout 3 --max-time 10 URL   getent hosts NAME                                  |
| Structured data | jq -r expression file.json   yq -r expression file.yaml                                              |
| Scheduling      | crontab -e   systemctl list-timers   journalctl -u unit                                              |

# Volume 2 Progress Tracker

| Unit | Topic                                                   | Status                                                                                                             | Completion Date |
|------|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------|
| 1    | Shell, Bash and the Automation Mindset                  | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 2    | Script Structure, Comments and Execution Modes          | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 3    | Variables, Quoting and Environment                      | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 4    | User Input, Arguments and Options                       | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 5    | Arithmetic and Numeric Validation                       | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 6    | Conditions, Tests and Decision Making                   | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 7    | case Statements and State Handling                      | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 8    | Loops and Repetition                                    | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 9    | Functions, Scope and Reusable Design                    | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 10   | Arrays and Associative Arrays                           | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 11   | Redirection, Pipes and File Descriptors                 | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 12   | Text Processing with grep, sed, awk, cut, sort and uniq | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 13   | Regular Expressions and Pattern Matching                | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 14   | Exit Codes, Error Handling and Strict Mode              | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____           |
| 15   | Traps, Signals and Cleanup                              | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress                                       | _____           |

|    |                                                           |                                                                                                                    |       |
|----|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-------|
|    |                                                           | <input type="checkbox"/> Completed                                                                                 |       |
| 16 | Logging, Timestamps and Observability                     | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 17 | Files, Directories and Safe Configuration Changes         | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 18 | Network Automation with curl, DNS and Timeouts            | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 19 | JSON, YAML and APIs with jq and yq                        | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 20 | Scheduling with cron and systemd Timers                   | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 21 | Remote Execution, SSH and Fleet Safety                    | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 22 | Packaging, Portability and Dependencies                   | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 23 | Security for Shell Scripts                                | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 24 | Testing Bash with ShellCheck, shfmt and Bats              | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 25 | Debugging and Performance Analysis                        | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 26 | Backup, Retention and Recovery Automation                 | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 27 | Deployment Automation Patterns                            | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 28 | CI/CD Integration for Shell Scripts                       | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 29 | Capstone Project: Production Health and Readiness Toolkit | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |
| 30 | Capstone Project: Backup and Restore Orchestrator         | <input type="checkbox"/> Not started<br><input type="checkbox"/> In progress<br><input type="checkbox"/> Completed | _____ |